



# BACHELOR'S THESIS TEMPLATE - TITLE OF PROJECT

Bachelor's Thesis

Arthur Ernst Henrik Reuss, s5582253, a.e.h.reuss@student.rug.nl,  
Supervisors: Gido van de Ven

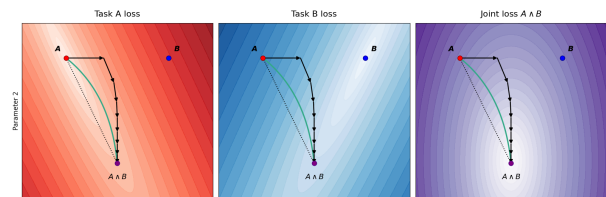
**Abstract:**

## 1 Introduction

Deep neural networks have achieved remarkable success in learning complex representations from static, independent, and identically distributed datasets. However, a significant vulnerability emerges when transitioning from static training environments to continuous, sequential learning: neural networks tend to greedily overwrite previously acquired knowledge to accommodate new information. This phenomenon, known as catastrophic forgetting [16, 21], represents a fundamental hurdle in machine learning. Standard approaches to accommodating new data frequently rely on offline retraining from scratch, a computationally expensive practice that remains impractical for systems requiring dynamic adaptation. Addressing this limitation forms the core challenge of Continual Learning (CL), which centers on navigating the stability-plasticity dilemma: the model must maintain sufficient plasticity to learn new data distributions while ensuring adequate stability to preserve past representations [17]. To address this, several techniques have been developed, including replay, parameter regularization, functional regularization, optimization-based approaches, context-dependent processing, and template-based classification [23]. Although these strategies have proven effective in mitigating long-term forgetting, they have historically been evaluated primarily after the completion of discrete tasks [13]. Recent advancements in continuous evaluation protocols have exposed an overlooked aspect of this task-oriented paradigm. By evaluating networks during the learning phase itself, Lange et al. [13] identified the *Stability Gap*, a transient but severe collapse in past-task accuracy that occurs immediately upon the introduction of a new task.

The discovery of the stability gap raised a critical question: could this transient collapse be eliminated by better approximating the joint loss over all tasks? Hess et al. [9] demonstrated that the stability gap persists even under *incremental joint training*, where the model has access to all training data from all tasks at every iteration. This implies: the stability gap cannot be eliminated by improving the loss function alone. If a perfect objective, the true joint loss over all data, still produces transient forgetting, then the problem lies not in *what* is being optimized but in *how* the optimization proceeds.

To understand why the optimization trajectory causes the stability gap even with a perfect objective, we must examine the fundamental geometry of sequential optimization in neural networks. Figure 1.1 presents a conceptual visualization of the optimization trajectory when transitioning from a previously learned task (Task A) to a new task (Task B). While this is a 2D simplification of a highly complex, high-dimensional neural network loss landscape, it captures the essential optimization dynamics at play.



**Figure 1.1: A conceptual visualization of the optimization trajectory across Task A, Task B, and the Joint Loss landscapes. The dotted line represents the ideal low-loss linear path. The green curve shows the theoretical steepest descent.**

The figure displays three corresponding loss landscapes: Task A individually (left), Task B individually

ally (center), and the mathematically perfect Joint Loss (right). The network begins at point  $A$ , representing a state of convergence where the model has successfully minimized the loss for Task A. The ultimate objective of continual learning is to navigate from point  $A$  to the optimal joint minimum, denoted as  $A \cap B$ .

If one were to perfectly approximate the Joint Loss and execute standard Stochastic Gradient Descent (SGD) using infinitely small step sizes, the optimizer would theoretically follow the green curve. Pulled by the steepest gradients of the joint landscape, this theoretical trajectory naturally curves outward. It systematically pulls the model out of Task A’s safe minimum and through a region of higher error before eventually converging at  $A \cap B$ , inherently inducing a transient stability gap in Task A performance.

However, there exists an alternative route: the dotted line. Recent work on linear mode connectivity has revealed that different local optima found by SGD are often connected by simple paths of non-increasing loss [5, 8]. This observation yields two critical insights. First, merely tying the practical optimization trajectory closer to the theoretical steepest-descent path (green curve) cannot eliminate the stability gap. Second, the optimizer must be actively constrained to navigate the direct, low-loss linear corridor (dotted line) from  $A$  to  $A \cap B$ . This reframes the central challenge: *what geometric information enables navigation along this low-loss path, and why do existing methods fail to exploit it?*

We propose a curvature-aware optimization framework that addresses this question directly. Our method decomposes each optimization step into two orthogonal objectives: ensuring sufficient progress toward the joint minimum via a cone constraint on admissible step directions, and minimizing damage to previously learned tasks via local Hessian information that identifies low-curvature directions. By navigating along the flattest available directions within the cone of acceptable joint-loss descent, our approach systematically follows the low-loss corridor illustrated by the dotted line in Figure 1.1, thereby mitigating the stability gap at its geometric source. The remainder of this thesis develops this idea in detail. Section 2 reviews the theoretical foundations that motivate this approach—mode connectivity, the geometric limitations of existing trajectory-control methods, and

the role of second-order information—before Section 4 presents the full algorithm.

## 2 Literature Review

### 2.1 Mode Connectivity: Low-Loss Paths Exist But Are Not Followed

Garipov et al. [8] and Draxler et al. [5] independently demonstrated that *nonlinear* paths—piecewise linear curves with a single bend or Bézier curves—can be constructed between independently trained minima, along which the loss remains nearly constant. The networks were trained on the same task but with different random initializations. However, this picture was later complicated by the observation that neural networks exhibit permutation symmetry: reordering the neurons in a hidden layer while simultaneously permuting the corresponding incoming and outgoing weights yields a functionally identical network. Two independently trained networks may therefore implement the same function yet appear distant in parameter space simply due to mismatched neuron orderings, causing naive linear interpolation to result in a spurious loss barrier. Entezari et al. [6] inferred that after finding and applying the permutation that best aligns the two networks, SGD solutions are in fact linearly connected with no barrier, suggesting that the barriers observed by Garipov et al. [8] and Draxler et al. [5] are largely labeling artifacts rather than intrinsic landscape geometry.

A stronger property, *linear* mode connectivity, emerges under the more favorable condition of shared initialization. Frankle et al. [7] showed that networks trained from the same initialization but with different SGD noise converge to minima that lie in the same low-loss basin, connected by a linear interpolation path with no error barrier. Crucially, shared initialization bypasses the permutation alignment problem entirely: since both networks begin from the same point, their neurons are never relabeled and no realignment is needed.

Mirzadeh et al. [18] extended this to the continual learning setting, showing that when a network pretrained on a first task is subsequently optimized on the joint loss over multiple tasks, the multitask solution remains linearly connected to the origi-

nal single-task minimum. Verwimp et al. [24] found that this also holds using replay-based continual learning, showing that approximating the joint loss is sufficient for the connectivity to be preserved. Kamath et al. [10] further confirmed that a linear low-loss path exists between consecutive minima in joint incremental learning, but that SGD does not follow it, instead passing through a high-loss region that manifests as the stability gap.

Three conditions facilitate linear mode connectivity in continual learning. The first is sufficient overparameterization [5], which is readily satisfied by modern deep networks. The second, and more critical for the continual learning setting, is shared initialization [7, 18, 24], which is naturally satisfied since each new task’s optimization begins from the weights obtained at the end of the previous task. The third is task similarity: Mirzadeh et al. [18] show that increasing task dissimilarity progressively breaks the linear path, with corrupted inputs or labels causing the multitask minimum to become disconnected from the previous task minimum. In standard continual learning, however, this condition is also naturally satisfied: the multitask solution always subsumes rather than replaces the previous task, meaning it must retain the features learned earlier and therefore remains geometrically close to the previous minimum. Lubana et al. [15] support this view from a different angle, showing that linear connectivity breaks precisely when two models rely on mechanistically different solutions exploiting different input features — a situation that the task entailment structure of continual learning is designed to avoid.

## 2.2 Why Existing Methods Fail: The Geometry of Trajectory Control

The preceding section establishes that low-loss linear paths between consecutive minima exist under conditions naturally met in continual learning. The critical remaining question is whether existing optimization methods can follow these paths. We now examine why they cannot.

Hess et al. [9] propose a conceptual framework that organizes continual learning methods along two orthogonal axes: methods that improve *what* to optimize—i.e., the loss function—and methods that improve *how* to optimize—i.e., the optimization trajectory. Experience replay and regularization-

based approaches such as EWC, SI, and Learning without Forgetting fall into the first category: they modify the objective to approximate the joint loss while leaving the optimizer unchanged. A smaller class of methods pursues trajectory control through gradient projection. As we demonstrate below, however, these methods rely on geometric information that becomes fundamentally uninformative along precisely the low-loss paths they are intended to follow.

### Gradient Episodic Memory and First-Order Projection.

Gradient Episodic Memory (GEM) [14] is the foundational trajectory-control method. GEM stores episodic memories of previous tasks and constrains each parameter update to prevent the loss on any stored task from increasing. Formally, when training on task  $t$ , the projected gradient  $\tilde{\mathbf{g}}$  is obtained by solving:

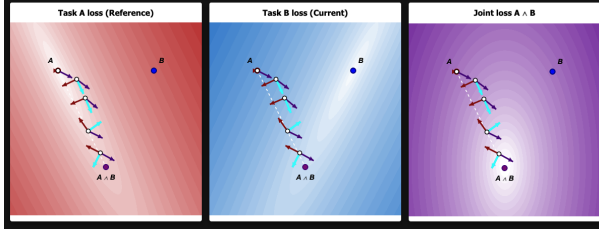
$$\tilde{\mathbf{g}} = \arg \min_{\tilde{\mathbf{g}}} \frac{1}{2} \|\mathbf{g}_t - \tilde{\mathbf{g}}\|_2^2 \quad \text{subject to} \quad \langle \tilde{\mathbf{g}}, \mathbf{g}_k \rangle \geq 0, \quad \forall k < t, \quad (2.1)$$

where  $\mathbf{g}_k$  denotes the gradient of the loss on the episodic memory of task  $k$ . Averaged GEM (A-GEM) [2] simplifies this by replacing the per-task constraints with a single averaged gradient. Related methods—OGD, GPM, and TRGP—similarly project updates onto the orthogonal complement of accumulated gradient subspaces.

All of these methods share a critical limitation: they rely exclusively on first-order information—gradients and their inner products—to identify safe update directions. This reliance fails in a predictable and structurally unavoidable way along the optimal path between task-specific minima.

**Two Failure Modes of GEM.** Consider the trajectory from the Task A minimum  $\theta_A$  toward the joint minimum  $\theta_{A \cap B}$ , depicted by the white dotted line in Figure 2.1. This trajectory exposes two distinct failure modes of first-order projection.

The first arises at the outset of Task B training. At initialization from  $\theta_A$ , the Task A gradient satisfies  $\mathbf{g}_A \approx \mathbf{0}$  by definition of a minimum, so GEM’s constraint  $\langle \tilde{\mathbf{g}}, \mathbf{g}_A \rangle \geq 0$  is trivial: almost any update direction has non-negative inner product with a near-zero vector. As shown in the top



**Figure 2.1: Geometric failure of GEM along the optimal path.** *Top:* Near the Task A minimum,  $\mathbf{g}_A \approx \mathbf{0}$ , rendering GEM’s constraint uninformative—the method reduces to unconstrained SGD. *Bottom:* Along the low-loss corridor toward the joint minimum, the reference gradient becomes nearly antiparallel to the optimal direction, causing GEM to project the update orthogonally away from the desired path.

panel of Figure 2.1, GEM therefore reduces to unconstrained SGD in this regime, permitting rapid displacement toward the Task B minimum before the constraint becomes active. By the time  $\|\mathbf{g}_A\|$  grows large enough to constrain updates meaningfully, catastrophic forgetting is already underway.

The second failure mode occurs farther along the path, as the optimizer approaches the joint minimum. Here, the reference gradient used by GEM becomes nearly antiparallel to the desired update direction. The resulting projection forces the step to be orthogonal to the low-loss corridor, actively steering the optimizer away from the joint minimum rather than toward it. This failure is illustrated by the points closest to the joint minimum, in Figure 2.1.

Hess et al. [9] propose combining GEM with experience replay (ER), replacing the Task B gradient with an approximation of the joint loss gradient. While this modification addresses the source of the optimization gradient, it does not resolve either failure mode: the reference constraint gradient  $\mathbf{g}_A$  remains near zero at the Task A minimum and becomes antiparallel to the joint gradient along the corridor, reproducing the same geometric pathologies. This is consistent with the empirical findings reported by those authors.

### Why First-Order Information Is Insufficient.

Both failure modes stem from the same underlying cause: the Task A gradient  $\mathbf{g}_A$  carries essentially no

information about the *transverse* structure of Task A’s loss basin. The gradient encodes only the directional derivative along the path away from  $\theta_A$ , which grows monotonically with displacement but reveals nothing about how steeply the loss rises in the perpendicular directions. Critically, a vanishing gradient does not distinguish between a flat valley floor and a narrow ridge—both exhibit  $\mathbf{g}_A \approx \mathbf{0}$  at the minimum, yet the former permits large transverse steps without loss increase while the latter does not.

The information needed to make this distinction is encoded in the *Hessian*  $H_A$ , the matrix of second-order partial derivatives describing the curvature of Task A’s loss surface in every direction. The eigen-spectrum of  $H_A$  reveals precisely what first-order constraints cannot: which directions correspond to flat valley floors (small eigenvalues, low curvature) and which to steep basin walls (large eigenvalues, high curvature). A step along a low-curvature direction incurs minimal forgetting; a step along a high-curvature direction causes rapid degradation—yet both are indistinguishable to a first-order constraint when  $\mathbf{g}_A \approx \mathbf{0}$ .

Prior work has recognized the relevance of curvature to continual learning in various indirect ways. Sharpness-aware methods seek flat minima with uniformly small Hessian eigenvalues [1, 4, 22], while training-regime methods use reduced learning rates to bias convergence toward wider basins [19]. However, no existing method incorporates the Hessian *directly* into the step-direction selection at each iteration during continual training. Our approach addresses this gap by using local curvature information to identify and follow low-loss corridors through the joint parameter landscape.

### 2.2.1 Second-Order Methods: Wrong Use of Curvature Information

If first-order methods fail geometrically, what about methods that already incorporate second-order information? Natural Continual Learning (NCL; Kao et al. 11) is the most sophisticated existing second-order approach and the closest in spirit to our proposed method. NCL adds a KL-divergence penalty to the loss, weighted by the Fisher information matrix  $F_A$  of the previous task:

$$\ell_{\text{NCL}} = \ell_{\text{new}} + \frac{\lambda}{2}(\theta - \mu_A)^\top F_A(\theta - \mu_A) \quad (2.2)$$

where  $\mu_A$  is the parameter vector at the Task A minimum. NCL uses a Kronecker-factored approximation to make the Fisher tractable and applies the inverse Fisher as a preconditioner to the gradient update.

This is a meaningful advance over first-order methods: the Fisher encodes curvature information that pure gradient methods lack, and the preconditioning encourages steps to follow low-curvature directions of the previous task’s loss. However, NCL differs from what is needed in one fundamental and one organizational sense.

**Global anchoring vs. local curvature.** NCL computes  $F_A$  once at the Task A minimum  $\mu_A$  and keeps it fixed throughout subsequent training. The penalty therefore measures displacement from a fixed anchor, weighted by curvature at that single point. But the loss landscape of a neural network is not a static paraboloid: the Hessian rotates and reshapes as the optimizer moves through parameter space. A direction that is flat at  $\mu_A$  may become steep midway to the joint minimum, and vice versa. Anchored at  $\mu_A$ , NCL cannot detect these changes. What is needed is *local* curvature information evaluated at the *current* parameter position, updated as the optimizer traverses the path between minima.

**Objective modification vs. trajectory control.** In the framework of Hess et al. [9], NCL belongs to the class of methods that improve *what* to optimize: it modifies the loss function by adding a Fisher-weighted quadratic penalty, thereby shifting the minimizer away from the true joint optimum by an amount that depends on  $\lambda$ . This means NCL does not purely control the *trajectory* through the joint loss landscape. Our approach instead targets trajectory control directly, using local curvature to identify and follow low-loss corridors without modifying the optimization objective.

Other second-order methods such as EWC and SI share the global-anchoring limitation: they fix importance weights at past parameter values and modify the objective rather than adapting to the local geometry of the path toward the joint minimum.

### 3 Research Questions

As established in Section 2, no existing continual learning method uses local second-order information for pure trajectory control: first-order methods lack curvature awareness, while second-order methods anchor their curvature estimates at past minima and modify the loss function rather than controlling the optimization path. The proposed curvature-aware cone-constrained optimization addresses this gap. This leads to the following research questions:

**RQ1: Stability Gap Reduction.** Does curvature-aware cone-constrained optimization reduce the stability gap compared to experience replay (ER), Gradient Episodic Memory (GEM), and Natural Continual Learning (NCL)?

What baselines make sense. Can I do all three?

**RQ1a: Cone Angle Sensitivity.** How sensitive is the stability gap reduction to the cone angle  $\alpha$ , and can a robust default value be identified across benchmarks?

**RQ1b: Computational Overhead.** What is the computational cost of the Lanczos-based curvature estimation relative to the baselines, and does the stability gap reduction justify the additional overhead?

### 4 The Proposed Approach

We propose a method that uses local second-order information for pure trajectory control, separating the optimization step into two orthogonal concerns: *ensuring sufficient progress toward the joint minimum* and *minimizing damage to previously learned tasks*. Unlike existing methods that modify the loss function (e.g., EWC, NCL) or rely on first-order gradient projections (e.g., GEM, A-GEM), our approach preserves the replay-approximated joint loss while controlling the optimization trajectory through local curvature information evaluated at the *current* parameter position.

**Prerequisites:** This strategy is predicated on the existence of a low-loss linear path between consecutive task minima. As established in Section 2, three conditions facilitate this connectivity: sufficient overparameterization, shared initialization, and adequate task similarity. The first two are naturally satisfied in the continual learning setting—



modern networks are heavily overparameterized, and each task’s optimization begins from the previous task’s solution. We assume the third condition holds for the standard benchmarks considered in our experiments (Section ??), consistent with the empirical findings of Mirzadeh et al. [18].

#### 4.1 Core Concept

The key insight from Section 2 is that near any previous task minimum, the per-task gradient is nearly zero and carries no directional information about the transverse structure of the loss basin, while the Hessian of the replay loss  $H_{\text{replay}}$  encodes which directions are safe (low curvature) and which are dangerous (high curvature) across all previously learned tasks simultaneously. Rather than using per-task gradients to constrain the step as GEM does, we employ a three-component strategy:

1. Define a **cone constraint** around the joint gradient  $\mathbf{g}_{\text{joint}}$  to ensure sufficient progress on the joint loss.
2. Within this cone, **deflect the step away from high-curvature directions** of the replay loss, identified via a low-rank approximation to  $H_{\text{replay}}$  at the current  $\theta$ .
3. Use a **trust region** to control step size within the regime where the local quadratic approximation remains valid.

This approach keeps the optimization objective unchanged while controlling the trajectory through curvature-aware direction selection informed by local second-order information.

#### 4.2 Computing the Joint Gradient

At each optimization step, we compute the gradient of the replay-approximated joint loss:

$$\mathbf{g}_{\text{joint}} = \nabla_{\theta} (\ell_{\text{current}}(\theta; \mathcal{B}_t) + \ell_{\text{replay}}(\theta)) \quad (4.1)$$

where  $\mathcal{B}_t$  is a mini-batch from the current task  $t$ , and the replay loss aggregates contributions from all previous tasks:

$$\ell_{\text{replay}}(\theta) = \sum_{k=1}^{t-1} \ell_k(\theta; \mathcal{M}_k) \quad (4.2)$$

with  $\mathcal{M}_k$  denoting the episodic memory buffer for task  $k$ . The memory buffers are populated using reservoir sampling with a fixed total budget of  $M$  samples distributed across all previous tasks. The gradient  $\mathbf{g}_{\text{joint}}$  represents the direction of steepest descent for the joint loss and serves as the central axis around which we define our cone constraint.

#### 4.3 The Cone Constraint

At each step, we define a cone  $\mathcal{C}$  of half-angle  $\alpha$  around the normalized joint gradient  $\hat{\mathbf{g}}_{\text{joint}} = \mathbf{g}_{\text{joint}} / \|\mathbf{g}_{\text{joint}}\|$ :

$$\mathcal{C} = \left\{ \mathbf{d} \in \mathbb{R}^P : \frac{\mathbf{d}^\top \hat{\mathbf{g}}_{\text{joint}}}{\|\mathbf{d}\|} \geq \cos(\alpha) \right\} \quad (4.3)$$

Any unit direction  $\mathbf{d} \in \mathcal{C}$  satisfies  $\mathbf{d}^\top \hat{\mathbf{g}}_{\text{joint}} \geq \cos(\alpha)$ , guaranteeing that at least a fraction  $\cos(\alpha)$  of the step magnitude contributes to descent on the joint loss. For example, with  $\alpha = 45$ , at least  $\cos(45) \approx 0.707$  of each step is productive joint-loss descent, while the remaining freedom allows the optimizer to steer away from high-curvature directions.

The cone angle  $\alpha$  is a hyperparameter controlling the tradeoff between joint-loss progress and old-task protection:

- At  $\alpha = 0$ : The cone degenerates to a single ray  $\hat{\mathbf{g}}_{\text{joint}}$ , recovering standard gradient descent on the joint loss.
- At moderate  $\alpha$  (e.g., 30–60): The cone provides sufficient directional freedom for curvature-based deflection while ensuring steady progress.
- At  $\alpha = 90$ : The cone spans the entire descent hemisphere.

#### 4.4 Identifying Dangerous Directions via Lanczos and Hessian-Vector Products

To identify which directions within the cone are harmful to previously learned tasks, we require knowledge of the high-curvature directions of the replay loss at the *current*  $\theta$ . The Hessian of  $\ell_{\text{replay}}$  is:

$$H_{\text{replay}} = \nabla_{\theta}^2 \ell_{\text{replay}}(\theta) = \sum_{k=1}^{t-1} H_k(\theta) \quad (4.4)$$

where  $H_k(\theta) = \nabla_{\theta}^2 \ell_k(\theta; \mathcal{M}_k)$  is the per-task Hessian evaluated at the current  $\theta$ . A direction with high curvature under  $H_{\text{replay}}$  is harmful across the aggregate of all previous tasks; deflecting away from such directions therefore protects all previous tasks simultaneously. This distinguishes our method from anchored approaches such as NCL and EWC, which fix curvature estimates at past minima and cannot track how the loss landscape evolves as the optimizer moves.

**Hessian-vector products via Pearlmutter’s trick.** We never form  $H_{\text{replay}}$  explicitly. Instead, for any vector  $\mathbf{v} \in \mathbb{R}^p$ , we obtain the exact product  $H_{\text{replay}}\mathbf{v}$  at cost  $O(p)$  using the identity [20]:

$$H_{\text{replay}}\mathbf{v} = \nabla_{\theta} [(\nabla_{\theta} \ell_{\text{replay}})^{\top} \mathbf{v}] \quad (4.5)$$

which requires two backward passes through the computation graph of  $\ell_{\text{replay}}$  and no  $O(p^2)$  storage. Because  $\ell_{\text{replay}}$  is a sum over all previous tasks, autograd accumulates contributions from every  $H_k$  in a single pass — the per-task decomposition in Equation 4.4 is implicit and requires no additional computation. Crucially, the gradient  $\nabla_{\theta} \ell_{\text{replay}}$  needed here is already available from the joint gradient computation in Equation 4.1.

**Lanczos iteration.** We apply the Lanczos algorithm [12] using the  $H_{\text{replay}}\mathbf{v}$  oracle above to build a low-dimensional approximation of the dangerous subspace. Starting from a random unit vector  $\mathbf{q}_1$ , the iteration constructs an orthonormal basis  $\{\mathbf{q}_j\}_{j=1}^k$  for the Krylov subspace  $\mathcal{K}_k = \text{span}\{\mathbf{q}_1, H_{\text{replay}}\mathbf{q}_1, \dots, H_{\text{replay}}^{k-1}\mathbf{q}_1\}$  via the three-term recurrence:

$$\mathbf{z}_j = H_{\text{replay}}\mathbf{q}_j \quad (4.6)$$

$$\alpha_j = \mathbf{q}_j^{\top} \mathbf{z}_j \quad (4.7)$$

$$\mathbf{z}_j \leftarrow \mathbf{z}_j - \alpha_j \mathbf{q}_j - \beta_{j-1} \mathbf{q}_{j-1} \quad (4.8)$$

$$\beta_j = \|\mathbf{z}_j\|, \quad \mathbf{q}_{j+1} = \mathbf{z}_j / \beta_j \quad (4.9)$$

After  $k$  steps this produces a tridiagonal matrix  $T_k \in \mathbb{R}^{k \times k}$  whose eigendecomposition  $T_k = Q\Lambda Q^{\top}$  yields approximate eigenpairs  $(\lambda_i, \mathbf{v}_i)$  of  $H_{\text{replay}}$ . Lanczos converges to the *extreme* eigenvalues fastest, so with  $k \approx 10$ –20 matrix-vector products we obtain reliable approximations to the  $d$  largest eigenvalues and their eigenvectors. The total cost per optimization step is  $2k$  backward passes

through  $\ell_{\text{replay}}$ , regardless of the number of previous tasks  $t - 1$ .

We extract only the top- $d$  eigenvectors  $V_{\text{danger}} = [\mathbf{v}_1, \dots, \mathbf{v}_d]$  corresponding to the largest eigenvalues  $\lambda_1 \geq \dots \geq \lambda_d$ . The flat directions are never computed explicitly; they emerge implicitly as the complement of  $V_{\text{danger}}$ , as we now describe.

**Amortized Lanczos updates.** The dominant cost of the proposed method is the Lanczos iteration, which requires  $2k$  backward passes per optimization step. Naively applied, this makes each step approximately  $2k$  times more expensive than standard gradient descent. However, the dangerous subspace  $V_{\text{danger}}$  need not be recomputed at every step. The high-curvature directions of  $\ell_{\text{replay}}$  reflect the global geometry of the loss basin, which evolves slowly relative to the parameter updates taken within it — a single gradient step moves  $\theta$  by  $O(\eta)$ , while the Hessian eigenspectrum changes by  $O(\eta \|\nabla H_{\text{replay}}\|)$ , which is negligible for small step sizes. We therefore recompute  $V_{\text{danger}}$  only once every  $K$  optimization steps, reusing the cached subspace for the intervening deflation and cone projection steps. The joint gradient  $\mathbf{g}_{\text{joint}}$  and the trust ratio are still evaluated at every step, so progress guarantees and step size control remain intact. This amortization reduces the average per-step cost to  $2k/K$  additional backward passes beyond standard replay.

## 4.5 Curvature-Aware Direction Selection

Given  $V_{\text{danger}}$ , we seek a step direction  $\mathbf{d}^* \in \mathcal{C}$  that minimizes the quadratic form  $\mathbf{d}^{\top} H_{\text{replay}} \mathbf{d}$ , which quantifies the expected increase in the aggregate replay loss. The key observation is that this form is dominated by components along  $V_{\text{danger}}$ :

$$\begin{aligned} \mathbf{d}^{\top} H_{\text{replay}} \mathbf{d} &\approx \mathbf{d}^{\top} V_{\text{danger}} \Lambda_d V_{\text{danger}}^{\top} \mathbf{d} \\ &\quad + \underbrace{\mathbf{d}^{\top} V_{\text{rest}} \Lambda_{\text{rest}} V_{\text{rest}}^{\top} \mathbf{d}}_{\approx 0} \end{aligned} \quad (4.10)$$

where  $\Lambda_d = \text{diag}(\lambda_1, \dots, \lambda_d)$  and  $\Lambda_{\text{rest}}$  contains eigenvalues small enough to be neglected. Minimizing the quadratic form therefore reduces to minimizing the component of  $\mathbf{d}$  along  $V_{\text{danger}}$ , which we achieve by projection.

**Deflation step.** We project the joint gradient out of the dangerous subspace:

$$\mathbf{g}_{\text{deflated}} = \hat{\mathbf{g}}_{\text{joint}} - V_{\text{danger}} V_{\text{danger}}^\top \hat{\mathbf{g}}_{\text{joint}} \quad (4.11)$$

This removes the components of  $\hat{\mathbf{g}}_{\text{joint}}$  that lie along high-curvature directions, leaving a candidate step that naturally lives in the low-curvature subspace.

**Cone projection.** We then check whether  $\mathbf{g}_{\text{deflated}}$  satisfies the cone constraint. If

$$\frac{\mathbf{g}_{\text{deflated}}^\top \hat{\mathbf{g}}_{\text{joint}}}{\|\mathbf{g}_{\text{deflated}}\|} \geq \cos(\alpha) \quad (4.12)$$

we set  $\mathbf{d}^* = \mathbf{g}_{\text{deflated}} / \|\mathbf{g}_{\text{deflated}}\|$  directly. Otherwise, we interpolate back toward  $\hat{\mathbf{g}}_{\text{joint}}$  until the constraint is exactly met:

$$\mathbf{d}^* = \frac{(1 - \beta)\hat{\mathbf{g}}_{\text{joint}} + \beta\hat{\mathbf{g}}_{\text{deflated}}}{\|(1 - \beta)\hat{\mathbf{g}}_{\text{joint}} + \beta\hat{\mathbf{g}}_{\text{deflated}}\|} \quad (4.13)$$

where  $\hat{\mathbf{g}}_{\text{deflated}} = \mathbf{g}_{\text{deflated}} / \|\mathbf{g}_{\text{deflated}}\|$  and  $\beta \in [0, 1]$  is chosen so that  $\mathbf{d}^*$  lies exactly on the cone boundary. This fallback ensures that even when the joint gradient is strongly aligned with a dangerous direction — the regime of maximal tension between tasks — the optimizer still makes guaranteed progress on the joint loss. The frequency of this fallback is itself a diagnostic: frequent fallback indicates that the joint gradient and the safe subspace are structurally in conflict, which characterizes the hardest continual learning settings.

The full direction selection step costs  $O(dp)$ , dominated by the  $d$  projections, and has no iterative component beyond the Lanczos pass already completed.

## 4.6 Trust Region Step Size Control

The curvature-based direction selection in Section 4.5 is only meaningful within a neighborhood of  $\theta$  where the local quadratic approximation of  $\ell_{\text{replay}}$  remains accurate. We employ a trust region on the step size  $\eta$  to enforce this. Rather than constructing a second-order predicted change — which would require the Hessian of the full joint loss including the current task, which is not available — we use a first-order trust ratio evaluated on the

joint loss directly:

$$\rho = \frac{\ell_{\text{joint}}(\theta + \eta \mathbf{d}^*) - \ell_{\text{joint}}(\theta)}{\eta \mathbf{g}_{\text{joint}}^\top \mathbf{d}^*} \quad (4.14)$$

The denominator is the directional derivative of  $\ell_{\text{joint}}$  along  $\mathbf{d}^*$ , predicting the loss change under a purely linear model. When  $\rho \approx 1$ , the joint loss behaves linearly in the step direction and the local approximation is valid; when  $\rho \ll 1$  or  $\rho > 1$ , higher-order curvature is significant and the step is too large. Both  $\ell_{\text{joint}}(\theta + \eta \mathbf{d}^*)$  and  $\mathbf{g}_{\text{joint}}^\top \mathbf{d}^*$  reuse quantities already computed — the numerator requires one additional forward pass on the current mini-batch and replay buffer, while the denominator is an inner product of previously computed quantities. We then update the radius according to standard trust region rules [3]:

$$r_{\text{new}} = \begin{cases} 2r & \text{if } \rho > 0.75 \\ \text{(linear model accurate, expand)} & \\ r & \text{if } 0.25 \leq \rho \leq 0.75 \\ \text{(acceptable, maintain)} & \\ 0.5r & \text{if } \rho < 0.25 \\ \text{(nonlinearity significant, contract)} & \end{cases} \quad (4.15)$$

When the joint loss is locally linear the radius expands, permitting larger curvature-steered steps; when significant nonlinearity is detected the radius contracts, keeping the optimizer within the regime where the Lanczos-based dangerous subspace estimate is geometrically meaningful. This is especially important as the optimizer traverses the corridor between minima, where the Hessian rotates and the local curvature changes — precisely the regime where fixed-anchor methods such as NCL and EWC lose fidelity.

## 4.7 Full Algorithm

Algorithm 4.1 summarizes the complete per-step procedure. The dominant cost is  $2k$  backward passes for the Lanczos iteration, where  $k \approx 10$ –20. The direction selection adds  $O(dp)$  arithmetic, and the trust ratio requires one additional forward pass to evaluate  $\ell_{\text{joint}}$  at the new parameters. No  $O(p^2)$  objects are ever formed.



---

**Algorithm 4.1** Curvature-Aware Continual Learning (CACL)

---

**Require:** Current task loss  $\ell_t$ , replay buffers  $\{\mathcal{M}_k\}_{k < t}$ , cone angle  $\alpha$ , Lanczos steps  $k$ , dangerous subspace size  $d$ , trust radius  $r$

- 1: Compute  $\mathbf{g}_{\text{joint}}$  and  $\nabla_{\theta} \ell_{\text{replay}}$  via Equations 4.1–4.2 with `create_graph=True`
- 2: Run  $k$  Lanczos steps using Pearlmutter  $H_{\text{replay}} \mathbf{v}$  products (Equation 4.5); extract top- $d$  eigenvectors  $V_{\text{danger}}$
- 3: Compute deflated gradient  $\mathbf{g}_{\text{deflated}}$  via Equation 4.11
- 4: **if**  $\mathbf{g}_{\text{deflated}}^{\top} \hat{\mathbf{g}}_{\text{joint}} / \|\mathbf{g}_{\text{deflated}}\| \geq \cos(\alpha)$  **then**
- 5:    $\mathbf{d}^* \leftarrow \mathbf{g}_{\text{deflated}} / \|\mathbf{g}_{\text{deflated}}\|$
- 6: **else**
- 7:   Find  $\beta$  such that  $\mathbf{d}^*$  in Equation 4.13 satisfies cone constraint exactly
- 8: **end if**
- 9: Compute step size  $\eta \leq r$ ; apply step  $\theta \leftarrow \theta + \eta \mathbf{d}^*$
- 10: Evaluate  $\ell_{\text{joint}}(\theta + \eta \mathbf{d}^*)$  via one forward pass; compute trust ratio  $\rho$  via Equation 4.14 using  $\mathbf{g}_{\text{joint}}^{\top} \mathbf{d}^*$ ; update  $r$  via Equation 4.15

---

## 4.8 Connection to the Theoretical Framework

Every component of the algorithm maps directly onto a claim established in Section 2. The cone constraint uses  $\mathbf{g}_{\text{joint}}$  rather than any per-task gradient as its axis, directly addressing the failure mode where individual task gradients  $\approx \mathbf{0}$  render GEM’s constraints trivial. The Lanczos-based curvature estimate uses  $H_{\text{replay}}$  evaluated at the current  $\theta$  at every step, addressing the stale-anchor limitation shared by NCL, EWC, and SI, while simultaneously aggregating curvature information across all previous tasks at the cost of a single Lanczos pass. The deflation step operationalizes the theoretical distinction between flat valley floors and narrow ridges that first-order methods cannot make. Finally, by leaving  $\ell_{\text{joint}}$  unmodified, the method belongs to the trajectory-control class in the framework of Hess et al. [9], avoiding the bias toward a shifted optimum that penalty-based methods incur.

## References

- [1] Ang Bian, Wei Li, Hangjie Yuan, Chengrong Yu, Mang Wang, Zixiang Zhao, Aojun Lu, Pengliang Ji, and Tao Feng. Make continual learning stronger via c-flat, 2024. URL <https://arxiv.org/abs/2404.00986>.
- [2] Arslan Chaudhry, Marc’Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. Efficient lifelong learning with a-gem, 2019. URL <https://arxiv.org/abs/1812.00420>.
- [3] Andrew R. Conn, Nicholas I. M. Gould, and Philippe L. Toint. *Trust-Region Methods*. MPS-SIAM Series on Optimization. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2000. ISBN 978-0-89871-460-9. doi: 10.1137/1.9780898719857.
- [4] Danruo Deng, Guangyong Chen, Jianye Hao, Qiong Wang, and Pheng-Ann Heng. Flattening sharpness for dynamic gradient projection memory benefits continual learning, 2021. URL <https://arxiv.org/abs/2110.04593>.
- [5] Felix Draxler, Kambis Veschgini, Manfred Salmhofer, and Fred A. Hamprecht. Essentially no barriers in neural network energy landscape, 2019. URL <https://arxiv.org/abs/1803.00885>.
- [6] Rahim Entezari, Hanie Sedghi, Olga Saukh, and Behnam Neyshabur. The role of permutation invariance in linear mode connectivity of neural networks, 2022. URL <https://arxiv.org/abs/2110.06296>.
- [7] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel M. Roy, and Michael Carbin. Linear mode connectivity and the lottery ticket hypothesis, 2020. URL <https://arxiv.org/abs/1912.05671>.
- [8] Timur Garipov, Pavel Izmailov, Dmitrii Podoprikin, Dmitry Vetrov, and Andrew Gordon Wilson. Loss surfaces, mode connectivity, and fast ensembling of dnns, 2018. URL <https://arxiv.org/abs/1802.10026>.
- [9] Timm Hess, Tinne Tuytelaars, and Gido M. van de Ven. Two complementary perspectives to continual learning: Ask not only what to

- optimize, but also how, 2024. URL <https://arxiv.org/abs/2311.04898>.
- [10] Sandesh Kamath, Albin Soutif-Cormerais, Joost van de Weijer, and Bogdan Raducanu. The expanding scope of the stability gap: Unveiling its presence in joint incremental learning of homogeneous tasks, 2024. URL <https://arxiv.org/abs/2406.05114>.
- [11] Ta-Chu Kao, Kristopher T. Jensen, Gido M. van de Ven, Alberto Bernacchia, and Guillaume Hennequin. Natural continual learning: success is a journey, not (just) a destination, 2021. URL <https://arxiv.org/abs/2106.08085>.
- [12] Cornelius Lanczos. An iteration method for the solution of the eigenvalue problem of linear differential and integral operators. *J. Res. Natl. Bur. Stand. B*, 45:255–282, 1950. doi: 10.6028/jres.045.026.
- [13] Matthias De Lange, Gido van de Ven, and Tinne Tuytelaars. Continual evaluation for lifelong learning: Identifying the stability gap, 2023. URL <https://arxiv.org/abs/2205.13452>.
- [14] David Lopez-Paz and Marc’Aurelio Ranzato. Gradient episodic memory for continual learning, 2022. URL <https://arxiv.org/abs/1706.08840>.
- [15] Ekdeep Singh Lubana, Eric J. Bigelow, Robert P. Dick, David Krueger, and Hidenori Tanaka. Mechanistic mode connectivity, 2023. URL <https://arxiv.org/abs/2211.08422>.
- [16] Michael McCloskey and Neil J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *The Psychology of Learning and Motivation*, 24:104–169, 1989.
- [17] Martial Mermillod, Aurélie Bugaiska, and Patrick Bonin. The stability-plasticity dilemma: Investigating the continuum from catastrophic forgetting to age-limited learning effects. *Frontiers in psychology*, 4:504, 08 2013. doi: 10.3389/fpsyg.2013.00504.
- [18] Seyed Iman Mirzadeh, Mehrdad Farajtabar, Dilan Gorur, Razvan Pascanu, and Hassan Ghasemzadeh. Linear mode connectivity in multitask and continual learning, 2020. URL <https://arxiv.org/abs/2010.04495>.
- [19] Seyed Iman Mirzadeh, Mehrdad Farajtabar, Razvan Pascanu, and Hassan Ghasemzadeh. Understanding the role of training regimes in continual learning. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 7308–7320. Curran Associates, Inc., 2020. URL [https://proceedings.neurips.cc/paper\\_files/paper/2020/file/518a38cc9a0173d0b2dc088166981cf8-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2020/file/518a38cc9a0173d0b2dc088166981cf8-Paper.pdf).
- [20] Barak Pearlmutter. Fast exact multiplication by the hessian. *Neural Computation*, 6:147–160, 01 1994. doi: 10.1162/neco.1994.6.1.147.
- [21] Roger Ratcliff. Connectionist models of recognition memory: Constraints imposed by learning and forgetting functions. *Psychological Review*, 97:285–308, 04 1990. doi: 10.1037/0033-295X.97.2.285.
- [22] Guangyuan Shi, Jiaxin Chen, Wenlong Zhang, Li-Ming Zhan, and Xiao-Ming Wu. Overcoming catastrophic forgetting in incremental few-shot learning by finding flat minima, 2021. URL <https://arxiv.org/abs/2111.01549>.
- [23] Gido M. van de Ven, Nicholas Soures, and Dhireesha Kudithipudi. *Continual learning and catastrophic forgetting*, page 153–168. Elsevier, 2025. ISBN 9780443157554. doi: 10.1016/b978-0-443-15754-7.00073-0. URL <http://dx.doi.org/10.1016/B978-0-443-15754-7.00073-0>.
- [24] Eli Verwimp, Matthias De Lange, and Tinne Tuytelaars. Rehearsal revealed: The limits and merits of revisiting samples in continual learning, 2021. URL <https://arxiv.org/abs/2104.07446>.